

APPLICATION OF ARTIFICIAL INTELLIGENCE IN IDENTIFYING VULNERABILITIES IN LOCAL AREA NETWORKS WITH FIREWALL INTEGRATION

João Vitor Ielen¹ Vinicius Mota Favaro²

Advisor: Prof. Michel Cesar dos Santos, Centro Universitário UniOpet

Academic approval: 19 November 2025. This date records the approval of the associated undergraduate thesis and is not a journal acceptance date.

ABSTRACT

Interpreting network-scan output is labor-intensive and may lead to overstatement when an observed service is treated as a confirmed vulnerability. This paper presents a defensive pipeline that parses authorized Nmap XML scans, inventories live hosts and open services, applies configuration-review rules, and generates JSON, Markdown, and self-contained HTML reports. The artifact defended in 2025 also included optional local language-model analysis and Zabbix integration; active firewall integration remained an architectural proposal. Deterministic evidence is conceptually separated from generated narrative. The reported evaluation comprises three scenarios, 24 hosts, and 21 open ports, in which seven review items were recorded. No independent ground-truth matrix was preserved to confirm false-positive counts or calculate detection metrics. The sample is too small to support universal claims, and open ports alone do not establish exploitability. Results indicate that the approach is useful for inventory, triage, and communication, provided that generated output is advisory and findings are validated against asset ownership, network position, product identity, version evidence, and configuration.

Keywords: network security; artificial intelligence; Nmap; large language model; defensive automation.

1 INTRODUCTION

The expansion of connected business processes has increased both operational efficiency and the number of services that organizations must inventory and protect. Network scans can reveal active hosts, open ports, service fingerprints, and operating-system guesses, but those observations require contextual interpretation. An open port is a communication endpoint, not proof of a vulnerability. Its security relevance depends

¹Undergraduate student in Software Engineering, Centro Universitário UniOpet, Curitiba, Brazil.

²Undergraduate student in Software Engineering, Centro Universitário UniOpet, Curitiba, Brazil.

on the actual product, version, configuration, reachability, authentication, encryption, business purpose, and compensating controls.

Nmap is widely used for network discovery and service fingerprinting (Lyon, 2009; Nmap Project, 2025). Its XML output is suitable for automated processing, yet a scan of even a modest network may produce enough detail to burden a small security team. Artificial intelligence, particularly large language models (LLMs), may assist with summarization and explanation. However, generated text can also invent CVEs, conflate exposure with exploitability, or recommend disruptive actions.

This study addresses the following question: *How can artificial-intelligence techniques support the identification, contextualization, and responsible mitigation of security risks observed in local networks?* The proposed answer is a hybrid architecture in which deterministic code owns evidence collection and rule matching, while an optional local LLM provides advisory narrative. The contribution is not a replacement for vulnerability management. It is an explainable bridge between scan evidence, human review, and defensive tooling.

The paper makes four contributions: (1) a reproducible parser and normalized inventory; (2) explicit review rules whose outputs are marked as unconfirmed; (3) a local-first architecture for LLM-assisted contextualization; and (4) a critical evaluation that distinguishes results supported by the sample from broader claims requiring further evidence.

2 BACKGROUND

2.1 Network discovery and vulnerability evidence

Nmap supports host discovery, port scanning, version detection, and probabilistic operating-system fingerprinting (Lyon, 2009). Fingerprints can be incomplete or wrong. A reliable association with a Common Vulnerabilities and Exposures identifier requires normalized product identity, affected-version evaluation, source freshness, and sometimes configuration information from an authenticated assessment. The National Vulnerability Database provides structured vulnerability information, but a service banner is not by itself a sufficient matching key (National Institute of Standards and Technology, 2025).

Network controls such as segmentation, firewalls, secure administration paths, and logging operate together (Kurose & Ross, 2021; National Institute of Standards and Technology, 2020). Automating a firewall response can shorten exposure, but an incorrect rule can disrupt legitimate services. Safe automation therefore needs approval, minimal scope, expiration, logging, and rollback.

2.2 Local language models in security analysis

Transformer-based LLMs generate text conditioned on prompts and supplied evidence. Llama-family models can be executed on local hardware (AI at Meta, 2024), and Ollama exposes a local API for model management and inference (Ollama, 2025). Local execution can keep topology information away from third-party cloud services, but it does not remove model risk.

An LLM may explain why Telnet deserves review, summarize affected endpoints, or organize validation questions. It should not independently assert exploitability, certify compliance, or issue blocking commands. The NIST AI Risk Management Framework emphasizes governance, measurement, and ongoing management of AI risks (National Institute of Standards and Technology, 2023). In this architecture, generated narrative remains traceable to deterministic input and subject to human review.

3 SYSTEM DESIGN

3.1 Pipeline

The system is divided into acquisition, normalization, deterministic review, contextual analysis, and presentation/integration. An operator first obtains an authorized Nmap XML scan. The parser retains hosts marked as up, selects an IP address, and extracts hostname, OS guess, and ports marked as open. For each port it records protocol, service name, product, and version.

The review engine compares service names and conventional ports with a small documented rule set. It covers cleartext administration and file transfer, web transport review, SMB/NetBIOS, database listeners, remote desktop, VNC, SSH hardening, and mail transport. A result contains the endpoint, observed fingerprint, triage severity, evidence, recommendation, and the explicit field `confirmed_vulnerability=false`.

Figure 1 summarizes the evidence-preserving processing pipeline, from the authorized Nmap XML input to human validation and defensive integration.

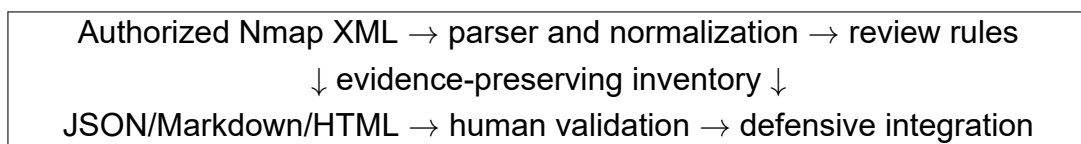


Figure 1 – Evidence-preserving processing pipeline.

Source: Prepared by the authors (2025).

3.2 Contextual analysis

The pre-defense artifact implemented three prompt families: technical summary, authorized test-scenario discussion, and compliance-oriented review. Its prompts supplied aggregate findings and requested a technical narrative. Model output did not modify the inventory, but the implementation did not enforce a structured output schema or independently verify the generated claims.

This distinction is essential. Python code determines what was parsed and which transparent rule matched. The LLM converts the structured result into human-readable context. Any future retrieval from authoritative sources should attach provenance and affected-version logic before a CVE is displayed.

A technical-analysis prompt summarized the attack surface and requested priorities. A controlled-assessment prompt discussed plausible attack paths for an authorized laboratory. A compliance-review prompt related findings to frameworks, although its output could not constitute certification. For all three, temperature and nucleus-sampling settings affect linguistic variability rather than factual reliability.

Model execution should be recorded as part of the experiment: model identifier, quantization, context length, generation parameters, prompt template, response, hardware, and elapsed time. Schema-constrained output and low-variance settings improve testability. Scan-controlled strings must be delimited as untrusted data because a hostname or banner could contain prompt-like instructions.

3.3 Reporting and integration

JSON supports machine processing, Markdown supports review and version control, and a self-contained HTML dashboard supports distribution without a web server. Nmap-controlled text is escaped before HTML rendering. The current Zabbix JSON-RPC integration uses environment-based credentials, finite timeouts, and TLS verification by default (Zabbix SIA, 2025). Export must be explicitly enabled.

Repository history provides two distinct software cuts (Ielen & Favaro, 2025). Commit dd63d4c, produced before the defense, implemented optional Ollama inference, a static service-to-CVE table, risk and compliance heuristics, the dashboard, and Zabbix export. Neither that cut nor the current code applies firewall rules. The current hardened core, introduced in b191269, removes LLM inference, CVE mapping, and heuristic scores while preserving inventory, transparent review rules, reports, and explicit Zabbix export. Code presence demonstrates implementation, not effectiveness or external validity.

3.4 Responsible firewall integration

A safe firewall workflow separates recommendation from authority. The analysis component should produce a proposal that identifies source, destination, protocol, port, rationale, duration, and affected assets. An authorized operator reviews the proposal before a vendor-specific adapter validates and applies it. Temporary rules require expiration, change logging, connectivity checks, and rollback. Protected management paths and critical services require explicit safeguards. The LLM must never hold direct firewall credentials.

3.5 Evidence states

To prevent semantic escalation, the design distinguishes four states: observed scan data; inferred fingerprint or context; deterministic review required; and independently confirmed vulnerability. Only a controlled follow-up assessment can assign the final state. Reports preserve those distinctions so that downstream consumers do not transform a conventional-port match into a CVE claim.

4 METHOD

The research is applied, exploratory, and technological. It is described retrospectively through the Design Science Research Methodology stages of problem identification, objectives, design and development, demonstration, evaluation, and communication (Peffer et al., 2007). Development followed incremental cycles inspired by Scrum (Schwaber & Sutherland, 2020). The reported evaluation is an exploratory demonstration rather than a controlled comparative experiment.

Scenario 1 represented a heterogeneous network with 19 active hosts and no deliberately planted weaknesses. Scenario 2 represented a simulated home network with three hosts and deliberately exposed SMB and HTTP services. Scenario 3 used two documentation-range servers configured with SSH and HTTPS to observe false-positive behavior in a hardened environment.

Table 1 – Reported evaluation scenarios.

ID	Environment	Hosts	Open ports	Review items	Time (s)
1	Heterogeneous network	19	9	3	222
2	Simulated vulnerable LAN	3	8	4	78
3	Hardened servers	2	4	0	52
Total		24	21	7	352

Source: Data reported in the original study (2025).

Precision and recall require an independently labeled ground truth. If TP , FP ,

and FN denote true positives, false positives, and false negatives, respectively, then

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}. \quad (1)$$

When the same rule database defines both expected and observed items, perfect scores primarily demonstrate conformance to that database. They cannot establish coverage of unknown services or the vulnerability landscape.

For reproducibility, each run should preserve the authorization and scope, Nmap parameters, source-XML hash, software commit, Python and dependency versions, generated reports, and an independently prepared ground-truth matrix. LLM evaluation additionally requires stored prompts and responses and a rubric covering evidence fidelity, technical correctness, usefulness, clarity, and unsupported assertions. Repeated runs are needed to characterize both timing and generation variability.

5 RESULTS AND DISCUSSION

Across the reported scenarios, 24 hosts and 21 open ports were processed, producing seven items in the scope of the evaluated rule set. The hardened scenario produced no items in the original evaluation. These figures were not reproduced during the present review because the complete source XML files, execution logs, and independent labels were not preserved. They support only a description of the reported demonstration.

The original manuscript reported approximately 7.5 seconds per host. The disclosed aggregate durations sum to 352 seconds; divided by 24 hosts, this is approximately 14.7 seconds per host. Even this corrected ratio should not be generalized because scanning time depends on port range, latency, retries, filters, scan method, and inference hardware. Likewise, a claim of linear scalability or a reported coefficient of determination cannot be reproduced without the underlying repeated measurements.

The evaluation is limited by the small sample and by potential circularity between rules and labels. Seven matched items do not establish broad vulnerability-detection performance. A service that is secure in context can still match a conventional-port rule, while an uncatalogued application vulnerability can be missed. The appropriate interpretation is *review prioritization*, not automated vulnerability confirmation.

The system's strongest property is semantic separation. Observable fields remain available in the report; rule-based findings explain why review is requested; and generated analysis is advisory. This design reduces the risk that plausible LLM text is mistaken for scanner evidence.

5.1 Scenario-level interpretation

In Scenario 1, the reported items involved HTTP and SMB exposure. Because the environment was not deliberately configured for the experiment, claiming no false positives would require independent validation of the policy and controls of each asset. The scenario demonstrates discovery but does not by itself provide ground truth.

Scenario 2 is a functional regression fixture. Its deliberately exposed services verify that parsing and expected rules operate consistently. Since the fixture is constructed from catalogued categories, it cannot measure performance on unknown weaknesses.

Scenario 3 illustrates semantic evolution. The original implementation reported no vulnerability for the SSH and HTTPS endpoints. The current hardened repository produces an SSH hardening review. This is not necessarily a false positive because the item explicitly asks for validation and states that it is not a confirmed vulnerability. Metrics must therefore score the precise output semantics rather than only count alerts.

5.2 Artifact-to-manuscript consistency

The source manuscript's larger dashboard, local Ollama inference, static CVE mappings, and risk/compliance heuristics are present in the pre-defense commit. No code was found that applied vendor firewall rules. The current repository implements a narrower defensive core: inventory, transparent review rules, static reports, and optional Zabbix export. This paper therefore recognizes the LLM as implemented in 2025 but not quantitatively validated, and recognizes firewall integration as proposed rather than implemented. Synchronizing claims with a tagged release is a prerequisite for a reproducible experimental claim.

5.3 Performance interpretation

Parser cost grows approximately with the number of XML host and port elements, while full-document parsing also creates a memory constraint. LLM inference, when enabled in an earlier prototype, was hardware-dependent and should be benchmarked separately. A future experiment should use fixed XML fixtures of increasing size, repeated measurements, and separate distributions for parsing, report generation, and inference.

6 THREATS TO VALIDITY AND LIMITATIONS

Internal validity is affected if expected outcomes were derived from the same static database used by the detector. External validity is limited by three scenarios, 24

hosts, and a narrow service set. Construct validity is threatened when open services, review items, and confirmed vulnerabilities are treated as equivalent. Reproducibility is limited because complete logs, repeated runs, exact prompts, hardware configuration, and independent expert ratings were not preserved in the source document.

Technical limitations include static rules, uncertain service fingerprints, absence of authenticated checks, lack of active exploitability validation, and full-document XML parsing. Markdown output requires robust escaping of untrusted values. Zabbix export should become idempotent. The LLM extension needs schema-constrained output, prompt-injection defenses, deterministic test fixtures, and evaluation by multiple blinded reviewers.

Ethically, only authorized scan data may be processed. Reports reveal sensitive infrastructure information and must be protected accordingly. Automated firewall changes should remain outside the trust boundary until approval, audit logging, expiration, and rollback have been implemented and tested.

7 CONCLUSION

This paper presented a hybrid, evidence-preserving approach to local-network security review. Deterministic parsing and transparent rules provide reproducibility; local LLM inference can improve explanation and prioritization; and multi-format reports support both analysts and integrations. The evaluation shows utility within a constrained rule set but does not support universal detection or exploitability claims.

The research question can therefore be answered conditionally: AI contributes most effectively as an advisory layer grounded in verifiable scan evidence. Vulnerability confirmation and mitigation still require authoritative data, environmental context, and human authorization. Future work should integrate version-aware NVD queries, expand independently labeled tests, evaluate LLM output blindly, stream large XML files, make Zabbix synchronization idempotent, and implement a controlled workflow for proposing, approving, and reversing firewall changes.

REFERENCES

- AI at Meta. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. Disponível em: <https://doi.org/10.48550/arXiv.2407.21783>. Acesso em: 3 ago. 2026.
- IELEN, J. V.; FAVARO, V. M. *TCC: Sistema de Análise de Segurança de Redes*. 2025. Repositório de código-fonte. Recorte pré-banca: commit dd63d4c; versão atual auditada: commit 9dd9026. Disponível em: <https://github.com/awakyy1/TCC>. Acesso em: 3 ago. 2026.
- KUROSE, J. F.; ROSS, K. W. *Computer Networking: A Top-Down Approach*. 8. ed.

Hoboken: Pearson, 2021.

LYON, G. F. *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Sunnyvale: Insecure.Com, 2009.

National Institute of Standards and Technology. *Security and Privacy Controls for Information Systems and Organizations*. [S.I.], 2020. NIST Special Publication 800-53, Revision 5. Disponível em: <https://doi.org/10.6028/NIST.SP.800-53r5>. Acesso em: 3 ago. 2026.

National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. [S.I.], 2023. NIST AI 100-1. Disponível em: <https://doi.org/10.6028/NIST.AI.100-1>. Acesso em: 3 ago. 2026.

National Institute of Standards and Technology. *National Vulnerability Database*. 2025. Disponível em: <https://nvd.nist.gov/>. Acesso em: 9 nov. 2025.

Nmap Project. *Nmap Reference Guide*. 2025. Disponível em: <https://nmap.org/book/man.html>. Acesso em: 9 nov. 2025.

Ollama. *Ollama Documentation*. 2025. Disponível em: <https://github.com/ollama/ollama>. Acesso em: 9 nov. 2025.

PEFFERS, K. et al. A design science research methodology for information systems research. *Journal of Management Information Systems*, v. 24, n. 3, p. 45–77, 2007. Disponível em: <https://doi.org/10.2753/MIS0742-1222240302>. Acesso em: 3 ago. 2026.

SCHWABER, K.; SUTHERLAND, J. *The Scrum Guide*. 2020. Disponível em: <https://scrumguides.org/scrum-guide.html>. Acesso em: 9 nov. 2025.

Zabbix SIA. *Zabbix Documentation*. 2025. Disponível em: <https://www.zabbix.com/documentation/>. Acesso em: 9 nov. 2025.